En este apartado, dado que ya conocemos el modelo y el enfoque que debemos aplicar, nos hemos centrado principalmente en probar diferentes variantes de dicho modelo y obtener resultados para su comparación.

Antes de implementar los modelos, utilizamos una función *clean_text*, pero que a diferencia de la del primer apartado de la práctica, sólo incluimos unos pocos pasos de normalización. Esto se hizo porque necesitamos que el texto incluya signos de puntuación e incluso mayúsculas.

El objetivo principal ha sido contar las apariciones de cada palabra en el texto. La idea es seleccionar como oraciones más importantes aquellas que contienen más palabras frecuentes a lo largo del texto. Para ello, hemos usado los métodos *CountVectorizer* y *TF-IDF*, que ya hemos trabajado previamente en esta práctica. Así, buscamos identificar las oraciones "más relevantes" y combinarlas para formar un resumen. Dependiendo del texto que queramos resumir, podemos ajustar el número de oraciones a incluir, o incluso configurar un resumen con una longitud que sea menor a un porcentaje específico del total del texto.

Aunque hemos probado diferentes enfoques, nos hemos centrado en aquellos que generan resúmenes extractivos basados únicamente en frecuencias. Dentro de estos, hemos probado un enfoque que se basa en la selección de palabras frecuentes, y otro similar en el que introducimos el concepto de *n-gramas*. Los *n-gramas* nos permiten trabajar con conjuntos de palabras de tamaño variable, lo que facilita una generalización del texto y un muestreo basado en frecuencias diferentes.

Para nuestra sorpresa, los resultados obtenidos con ambos enfoques fueron iguales, lo que nos lleva a la conclusión de que, al tratarse de un problema relativamente sencillo, no se presentaron grandes diferencias. El ajuste de los modelos se basó en la prueba y error, complementado con parte del contenido proporcionado en el temario subido a Moodle.

Finalmente, proporcionamos una explicación detallada de las dos implementaciones principales del notebook correspondiente.

Primera implementación

Este código implementa un sistema de resumen de texto utilizando dos enfoques: *CountVectorizer* y *TF-IDF*. A continuación se describe el proceso de manera resumida:

Primero, se importan las librerías necesarias, incluyendo herramientas de procesamiento de texto como *NLTK* (para tokenización de oraciones y manejo de stopwords). También se descargan los recursos necesarios de *NLTK*, como el tokenizador de oraciones y las *stopwords*.

El proceso de resumen se realiza mediante la función *generate_summary*. Esta función toma un texto y un diccionario de puntuaciones de palabras (obtenidas mediante *CountVectorizer* o *TF-IDF*) y calcula la relevancia de cada oración en función de la presencia de estas palabras. Las oraciones más relevantes se seleccionan para formar el resumen final.

Para generar las puntuaciones de las palabras:

- *CountVectorizer*: Convierte el texto en una representación de frecuencia de términos, es decir, cuenta cuántas veces aparece cada palabra en el texto.

- *TF-IDF*: Calcula la importancia de cada palabra en el contexto del texto, ponderando su frecuencia y su presencia en otros documentos.

Ambos métodos generan un diccionario con las palabras y sus respectivas puntuaciones.

Luego, la función *generate_summary* selecciona las oraciones más relevantes basadas en esas puntuaciones, generando dos resúmenes: uno con *CountVectorizer* y otro con *TF-IDF*.

Finalmente, se compara la longitud de los resúmenes generados por ambos métodos y se imprime el resultado.

Segunda implementación

Este código tiene como objetivo generar resúmenes de texto utilizando *n-gramas*, que son secuencias de palabras consecutivas en un texto. Para ello, emplea dos técnicas comunes de vectorización de texto: *CountVectorizer* y *TfidfVectorizer*. Estas herramientas convierten el texto en representaciones numéricas que capturan la frecuencia de aparición de las palabras y la importancia relativa de estas dentro del contexto del texto.

El proceso comienza con la tokenización del texto en oraciones usando la función *sent_tokenize* de *NLTK*. Luego, para crear los resúmenes, se genera un vector para las oraciones utilizando *CountVectorizer* y *TfidfVectorizer*, configurados para trabajar con

n-gramas de longitud variable, como *unigramas* (palabras individuales), *bigramas* (pares de palabras) y *trigramas* (tripletas de palabras). *CountVectorizer* cuenta la frecuencia de cada *n-grama* en el texto, mientras que *TfidfVectorizer* calcula el valor *TF-IDF*, que mide la relevancia de cada n-grama en función de su aparición en el texto y su rareza en comparación con todo el corpus. Es importante mencionar, que en función del tipo de n-grama, hemos decidio asignar unas ponderaciones de la importancia de las puntuaciones ofrecidas por cada modelo. La lógica detrás de las ponderaciones es:

- Para unigramas, la frecuencia es más importante porque las palabras individuales que se repiten suelen ser significativas.
- Para los bigramas, se da más peso al *TF-IDF* porque las frases de dos palabras son más específicas.
- Para trigramas, el *TF-IDF* es aún más importante porque las frases de tres palabras son muy específicas y su frecuencia es menos relevante

El factor de posición (0.8 + 0.2 * *position_weight*) da un pequeño *boost* a las oraciones que aparecen antes en el texto, ya que en muchos textos la información más importante suele aparecer al principio.

El resumen final se genera seleccionando las oraciones con las puntuaciones más altas, y el código permite comparar resúmenes usando diferentes configuraciones de *n-gramas*: unigramas, bigramas y trigramas. Además, se imprime la longitud de los resúmenes generados con cada tipo de *n-grama* para observar cómo varía el tamaño del resumen según el tipo de *n-grama* utilizado.